

SKILL WEEK-9

1 . To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, and Test Job Listings.

Aim : To implement a simple job-management system that can create background jobs, assign job IDs, maintain their process IDs and states, list active jobs, and update job status when processes finish.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");

    job_count++;
}

void update_jobs()
{
    ^G Help      ^O Write Out  ^W Where Is
    ^X Exit      ^R Read File  ^\ Replace
```

```
for (int i = 0; i < job_count; i++)
{
    if (strcmp(jobs[i].status, "Running") == 0)
    {
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result == jobs[i].pid)
        {
            if (WIFEXITED(status))
            {
                strcpy(jobs[i].status, "Done");
            }
            else
            {
                strcpy(jobs[i].status, "Terminated");
            }
        }
    }
}

void list_jobs()
{
    update_jobs();

    printf("\n");
    printf("===== ACTIVE JOBS =====\n");

    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
**^\
Replace**

^K Cut
^U Paste

```
GNU nano 7.2
    "Command");

    printf("-----\n");
    for (int i = 0; i < job_count; i++)
    {
        printf("%-8d %-10d %-15s %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].status,
            jobs[i].command);
    }

    if (job_count == 0)
    {
        printf("No jobs found.\n");
    }

    printf("-----\n");
}

void start_job(int seconds)
{
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n",
            getpid());

        sleep(seconds);

        printf("Child process %d completed.\n",
            getpid());
    }
}
```

^G Help **^O** Write Out **^W** Where Is **^K** Cut
^X Exit **^R** Read File **^N** Replace **^U** Paste

```
GNU nano 7.2
printf("==== JOB MANAGEMENT SYSTEM ====\n");
while (1)
{
    printf("\n");
    printf("1. Start Background Job\n");
    printf("2. List Jobs\n");
    printf("3. Exit\n");

    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        printf("Enter job duration in seconds: ");
        scanf("%d", &seconds);
        start_job(seconds);
    }
    else if (choice == 2)
    {
        list_jobs();
    }
    else if (choice == 3)
    {
        update_jobs();
        printf("\nExiting job manager...\n");
        break;
    }
    else
    {
        printf("Invalid choice.\n");
    }
}

return 0;
}
```

^G Help **^O** Write Out **^W** Where Is **^K** Cut
^X Exit **^R** Read File **^N** Replace **^U** Paste

Output :

```

root@keerthana: ~/job_practi × + ∨
root@keerthana:~# mkdir job_practical
root@keerthana:~# cd job_practical
root@keerthana:~/job_practical# pwd
/root/job_practical
root@keerthana:~/job_practical# ls
root@keerthana:~/job_practical# nano jobs.c
root@keerthana:~/job_practical# ls
jobs.c
root@keerthana:~/job_practical# cat jobs.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");

    job_count++;
}

void update_jobs()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(jobs[i].status, "Running") == 0)
        {
            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result == jobs[i].pid)
            {
                if (WIFEXITED(status))

```

```

root@keerthana: ~/job_practi
    if (WIFEXITED(status))
    {
        strcpy(jobs[i].status, "Done");
    }
    else
    {
        strcpy(jobs[i].status, "Terminated");
    }
}
}
}

void list_jobs()
{
    update_jobs();

    printf("\n");
    printf("===== ACTIVE JOBS =====\n");

    printf("%-8s %-10s %-15s %s\n",
           "Job ID",
           "PID",
           "Status",
           "Command");

    printf("-----\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%-8d %-10d %-15s %s\n",
               jobs[i].job_id,
               jobs[i].pid,

```

```

root@keerthana: ~/job_practi
        jobs[i].status,
        jobs[i].command);
    }

    if (job_count == 0)
    {
        printf("No jobs found.\n");
    }

    printf("-----\n");
}

void start_job(int seconds)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n",
               getpid());

        sleep(seconds);

        printf("Child process %d completed.\n",
               getpid());

        exit(0);
    }
}

```



root@keerthana: ~/job_practi



```
        exit(0);
    }
    else
    {
        char command[100];

        snprintf(command,
                 sizeof(command),
                 "sleep %d",
                 seconds);

        add_job(pid, command);

        printf("Job [%d] started with PID %d\n",
               job_count,
               pid);
    }
}

int main()
{
    int choice;
    int seconds;

    printf("==== JOB MANAGEMENT SYSTEM =====\n");

    while (1)
    {
        printf("\n");
        printf("1. Start Background Job\n");
        printf("2. List Jobs\n");
        printf("3. Exit\n");
```



root@keerthana: ~/job_practi



```
printf("Enter choice: ");
scanf("%d", &choice);

if (choice == 1)
{
    printf("Enter job duration in seconds: ");
    scanf("%d", &seconds);

    start_job(seconds);
}
else if (choice == 2)
{
    list_jobs();
}
else if (choice == 3)
{
    update_jobs();

    printf("\nExiting job manager...\n");
    break;
}
else
{
    printf("Invalid choice.\n");
}

return 0;
}
```

root@keerthana: ~/job_practi X + v

```
root@keerthana:~/job_practical# gcc jobs.c -o jobs
```

```
root@keerthana:~/job_practical# ls
```

```
jobs jobs.c
```

```
root@keerthana:~/job_practical# ./jobs
```

```
===== JOB MANAGEMENT SYSTEM =====
```

```
1. Start Background Job
```

```
2. List Jobs
```

```
3. Exit
```

```
Enter choice: 1
```

```
Enter job duration in seconds: 5
```

```
Job [1] started with PID 512
```

```
1. Start Background Job
```

```
2. List Jobs
```

```
3. Exit
```

```
Enter choice: Child process started. PID = 512
```

```
Child process 512 completed.
```

```
1
```

```
Enter job duration in seconds: 10
```

```
Job [2] started with PID 513
```

```
1. Start Background Job
```

```
2. List Jobs
```

```
3. Exit
```

```
Enter choice: Child process started. PID = 513
```

```
2
```

```
===== ACTIVE JOBS =====
```

Job ID	PID	Status	Command
1	512	Done	sleep 5
2	513	Running	sleep 10

Job ID	PID	Status	Command
1	512	Done	sleep 5
2	513	Running	sleep 10

1. Start Background Job
2. List Jobs
3. Exit

Enter choice: Child process 513 completed.

2

===== ACTIVE JOBS =====

Job ID	PID	Status	Command
1	512	Done	sleep 5
2	513	Done	sleep 10

1. Start Background Job
2. List Jobs
3. Exit

Enter choice: 9

Invalid choice.

1. Start Background Job
2. List Jobs
3. Exit

Enter choice: 3

Exiting job manager...

root@keerthana:~/job_practical# |

2 . To Design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, and Test Foreground Switching.

Aim : To implement foreground job control by identifying a target job, transferring terminal control to the selected process, resuming a stopped process, waiting for its completion, updating the job state, and testing foreground switching.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <termios.h>

int main()
{
    pid_t pid;
    int status;
    int shell_terminal;

    shell_terminal = STDIN_FILENO;

    printf("==== FOREGROUND JOB CONTROL ====\\n");

    /* Create a child process */
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    /* Child process */
    if (pid == 0)
    {
        printf("\\nChild process created.\\n");
        printf("Child PID: %d\\n", getpid());

        /* Create a new process group */
        setpgid(0, 0);

        /* Resume process if it was stopped */
        kill(getpid(), SIGCONT);

        printf("Child process is running in foreground...\\n");

        for (int i = 1; i <= 5; i++)
        {
            printf("Child working... %d/5\\n", i);
            sleep(1);
        }

        printf("Child process completed.\\n");
        exit(0);
    }
}
```

```

/* Parent process */
printf("\nParent process created child.\n");
printf("Child PID: %d\n", pid);

/* Put child in its own process group */
setpgid(pid, pid);

printf("\nIdentifying target job...\n");
printf("Target Job PID: %d\n", pid);

printf("\nTransferring terminal control to child...\n");

/*
 * Give terminal control to child process group.
 */
if (tcsetpgrp(shell_terminal, pid) == -1)
{
    perror("tcsetpgrp");
}

printf("Terminal control transferred.\n");

/* Wait for child */
printf("\nParent is waiting for foreground job...\n");
if (waitpid(pid, &status, WUNTRACED) == -1)
{
    perror("waitpid failed");
    return 1;
}

/* Check job state */
if (WIFSTOPPED(status))
{
    printf("\nJob stopped.\n");
    printf("Resuming foreground job...\n");
    kill(pid, SIGCONT);
    waitpid(pid, &status, 0);
}
if (WIFEXITED(status))
{
    printf("\nForeground job completed normally.\n");
    printf("Exit status: %d\n",
           WEXITSTATUS(status));
}

```

```

}

/* Give terminal back to parent */
printf("\nReturning terminal control to parent...\n");

if (tcsetpgrp(shell_terminal, getpgrp()) == -1)
{
    perror("tcsetpgrp");
}

printf("Terminal control returned to parent.\n");
printf("\nJob state updated: COMPLETED\n");
printf("\n==== FOREGROUND SWITCHING TEST COMPLETED =====\n");

return 0;
}

```

Output :

```
root@keerthana: ~/foregrour  X + v
root@keerthana:~/job_practical# cd ~
mkdir foreground_job
cd foreground_job
root@keerthana:~/foreground_job# pwd
/root/foreground_job
root@keerthana:~/foreground_job# nano foreground_job.c
root@keerthana:~/foreground_job# gcc foreground_job.c -o foreground_job
root@keerthana:~/foreground_job# ls
foreground_job  foreground_job.c
root@keerthana:~/foreground_job# ./foreground_job
===== FOREGROUND JOB CONTROL =====

Parent process created child.

Child process created.
Child PID: 530
Child process is running in foreground...
Child working... 1/5

Identifying target job...
Target Job PID: 530

Transferring terminal control to child...
Terminal control transferred.

Parent is waiting for foreground job...
Child working... 2/5
Child working... 3/5
Child working... 4/5
Child working... 5/5
Child process completed.

Foreground job completed normally.
```

```
root@keerthana: ~/foregrour  X + v
root@keerthana:~/foreground_job# ls
foreground_job  foreground_job.c
root@keerthana:~/foreground_job# ./foreground_job
===== FOREGROUND JOB CONTROL =====

Parent process created child.

Child process created.
Child PID: 530
Child process is running in foreground...
Child working... 1/5

Identifying target job...
Target Job PID: 530

Transferring terminal control to child...
Terminal control transferred.

Parent is waiting for foreground job...
Child working... 2/5
Child working... 3/5
Child working... 4/5
Child working... 5/5
Child process completed.

Foreground job completed normally.
Exit status: 0

Returning terminal control to parent...

[1]+  Stopped                  ./foreground_job
root@keerthana:~/foreground_job# |
```

3 . To analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior using a C program and Linux commands.

Aim : To study how signals are delivered to processes, how signal masks block and unblock signals, how process groups are identified, and how terminal signals such as `SIGINT` and `SIGTSTP` affect processes.

Program :

```
alekhya@Alekhyareddy: ~/OS × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received: Ctrl+C signal delivered.\n");

    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received: User-defined signal delivered.\n");

    else if (sig == SIGTERM)
        printf("\nSIGTERM received: Termination signal delivered.\n");
}

int main()
{
    sigset_t mask;
    pid_t pid = getpid();

    printf("Signal Behavior Analysis\n");
    printf("-----\n");

    printf("Process PID      : %d\n", pid);
    printf("Process Group ID : %d\n", getpgrp());
    printf("Parent PID       : %d\n", getppid());

    /* Register signal handlers */
    signal(SIGINT, signal_handler);
    signal(SIGUSR1, signal_handler);
    signal(SIGTERM, signal_handler);

    printf("\nSignal handlers registered.\n");
}
```

alekhya@Alekhyareddy: ~/OS × + ▾

GNU nano 7.2

```
signal(SIGUSR1, signal_handler);
signal(SIGTERM, signal_handler);

printf("\nSignal handlers registered.\n");

/* Create signal mask */
sigemptyset(&mask);
sigaddset(&mask, SIGINT);

printf("Blocking SIGINT for 10 seconds...\n");

if (sigprocmask(SIG_BLOCK, &mask, NULL) == -1)
{
    perror("sigprocmask");
    return 1;
}

printf("SIGINT is now blocked.\n");
printf("Press Ctrl+C during these 10 seconds.\n");

sleep(10);

printf("\nUnblocking SIGINT...\n");

if (sigprocmask(SIG_UNBLOCK, &mask, NULL) == -1)
{
    perror("sigprocmask");
    return 1;
}

printf("SIGINT is now unblocked.\n");

printf("\nWaiting for signals...\n");
printf("Use another terminal to send SIGUSR1 or SIGTERM.\n");

while (1)
{
    pause();
}

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

^T E
^J J

Output :

```
root@keerthana: ~/OSSP
root@keerthana:~/OSSP# cd ~/OSSP
root@keerthana:~/OSSP# ls
LICENSE      Week7  include  task1.c
Makefile     Week8  obj       task_signal_behavior
README.md    assets scripts  task_signal_behavior.c
Week5        bin    src       tests
Week6        docs   task1

root@keerthana:~/OSSP# ./task_signal_behavior
Signal Behavior Analysis
-----
Process PID      : 636
Process Group ID : 636
Parent PID       : 615

Signal handlers registered.
Blocking SIGINT for 10 seconds...
SIGINT is now blocked.
Press Ctrl+C during these 10 seconds.

Unblocking SIGINT...
SIGINT is now unblocked.

Waiting for signals...
Use another terminal to send SIGUSR1 or SIGTERM.

SIGUSR1 received: User-defined signal delivered.

SIGTERM received: Termination signal delivered.
|
```

4 . To implement resuming of stopped jobs by sending continue signals, updating job status, maintaining a job table, verifying execution, and testing background recovery.

Aim : To demonstrate how a stopped process can be resumed using `SIGCONT`, update its job status from STOPPED to RUNNING, and manage the job using a simple job table.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

#define STOPPED 1
#define RUNNING 2
#define COMPLETED 3

typedef struct
{
    pid_t pid;
    int job_id;
    int status;
} Job;

int main()
{
    Job job;
    pid_t pid;
    int status;

    printf("Job Control - Resume Stopped Job\n");
    printf("-----\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started.\n");
        printf("PID: %d\n", getpid());
        printf("Job is running...\n");

        while (1)
        {
            printf("Background job is executing...\n");
            sleep(2);
        }
    }

    }
```

Help **Write Out** **Where Is** **Cut**
Exit **Read File** **Replace** **Paste**


```
    }  
}  
  
/* Create job table entry */  
job.pid = pid;  
job.job_id = 1;  
job.status = RUNNING;  
  
printf("\nJob Table\n");  
printf("Job ID : %d\n", job.job_id);  
printf("PID      : %d\n", job.pid);  
printf("Status : RUNNING\n");  
  
sleep(5);  
  
/* Stop the job */  
printf("\nStopping job %d...\n", job.job_id);  
  
kill(job.pid, SIGSTOP);  
  
waitpid(job.pid, &status, WUNTRACED);  
  
if (WIFSTOPPED(status))  
{  
    job.status = STOPPED;  
  
    printf("Job ID : %d\n", job.job_id);  
    printf("PID      : %d\n", job.pid);  
    printf("Status : STOPPED\n");  
}  
  
/* Resume the stopped job */  
printf("\nSending SIGCONT to resume job...\n");  
  
if (kill(job.pid, SIGCONT) == -1)  
{  
    perror("SIGCONT");  
    return 1;  
}  
  
job.status = RUNNING;  
  
printf("Continue signal sent successfully.\n");  
printf("Job ID : %d\n", job.job_id);  
printf("PID      : %d\n", job.pid);  
printf("Status : RUNNING\n");  
|
```

```
    job.status = STOPPED;

    printf("Job ID : %d\n", job.job_id);
    printf("PID    : %d\n", job.pid);
    printf("Status : STOPPED\n");
}

/* Resume the stopped job */
printf("\nSending SIGCONT to resume job...\n");

if (kill(job.pid, SIGCONT) == -1)
{
    perror("SIGCONT");
    return 1;
}

job.status = RUNNING;

printf("Continue signal sent successfully.\n");
printf("Job ID : %d\n", job.job_id);
printf("PID    : %d\n", job.pid);
printf("Status : RUNNING\n");

/* Verify execution */
printf("\nVerifying job execution...\n");
sleep(5);

printf("Job %d is running again.\n", job.job_id);

/* Terminate job */
printf("\nTerminating background job...\n");

kill(job.pid, SIGTERM);

waitpid(job.pid, &status, 0);

job.status = COMPLETED;

printf("Job ID : %d\n", job.job_id);
printf("PID    : %d\n", job.pid);
printf("Status : COMPLETED\n");

printf("\nJob recovery test completed successfully.\n");

return 0;
}
```

Output :

```
root@keerthana: ~/OSSP
root@keerthana:~/OSSP# cd ~/OSSP
root@keerthana:~/OSSP# nano task_resume_job.c
root@keerthana:~/OSSP# gcc -Wall -Wextra task_resume_job.c -o task_resume_job
root@keerthana:~/OSSP# ./task_resume_job
Job Control - Resume Stopped Job
-----

Job Table
Job ID : 1
PID    : 720
Status : RUNNING
Child process started.
PID: 720
Job is running...
Background job is executing...
Background job is executing...
Background job is executing...

Stopping job 1...
Job ID : 1
PID    : 720
Status : STOPPED

Sending SIGCONT to resume job...
Continue signal sent successfully.
Job ID : 1
PID    : 720
Status : RUNNING

Verifying job execution...
Background job is executing...
Background job is executing...
Job 1 is running again.
```

```
root@keerthana: ~/OSSP
Child process started.
PID: 720
Job is running...
Background job is executing...
Background job is executing...
Background job is executing...

Stopping job 1...
Job ID : 1
PID    : 720
Status : STOPPED

Sending SIGCONT to resume job...
Continue signal sent successfully.
Job ID : 1
PID    : 720
Status : RUNNING

Verifying job execution...
Background job is executing...
Background job is executing...
Job 1 is running again.

Terminating background job...
Job ID : 1
PID    : 720
Status : COMPLETED

Job recovery test completed successfully.
root@keerthana:~/OSSP# |
```